

AI Evidence Engine & Project Forecasting

Wow Factor - Technical Explanation

A manager-facing project intelligence feature that combines real project data, deterministic forecasting, evidence quality, risk analysis, external engineering activity, and AI-generated explanations.

The important distinction

The wow factor is **not** simply that Gemini is connected to the system. The core value comes from the forecasting and evidence engine that calculates project health from real system data. Gemini is used afterward to explain the already-calculated results in clear language.

Project: Timesheets AI

Feature: Project-level delivery forecasting

Audience: Managers

Author: Nyasha

Date: September 2026

1. What the wow factor does

The Project Forecasting feature answers a practical management question: **Based on what is happening in the project right now, where is this project heading?**

Instead of showing only historical timesheet totals, the feature combines current project data to estimate remaining work, expected completion, budget usage, delivery risk and forecast confidence. It also shows which evidence was available and which evidence was missing, so the manager can judge how much trust to place in the forecast.

Input / Evidence	What it tells the forecast
Logged time	How many hours the team has already spent on the project.
Task status	How much work is complete and how much remains.
Task estimates	How many estimated hours remain on unfinished work.
Recent team velocity	The team's recent working pace, converted into hours per week.
Project dates	Whether the predicted completion date is ahead of or behind the planned schedule.
Budget hours	Whether projected total hours are likely to exceed the project's hour budget.
GitHub activity	Supporting evidence that development activity is occurring.
Jira activity	Supporting evidence from issue/task activity when the integration is available.

Example from the working system

For the Mobile App Development project, the system could report 27.7 used hours, 18 estimated remaining hours, a 45.7-hour projected total, a forecast completion date, project risk, and a confidence score based on how much supporting evidence was available.

2. How the forecast is calculated

The calculation is deliberately split into smaller steps. This makes the result easier to test, explain and defend during a demo.

Step	Calculation	Manager-facing result
1. Load project data	Read the project, active tasks, time entries and supporting evidence.	The forecast is based on the selected project only.
2. Used hours	Sum valid time-entry durations and convert seconds to hours.	Hours already consumed.
3. Task progress	Count total, completed and remaining tasks.	Completion percentage and remaining task count.
4. Remaining work	For each incomplete task: $\max(\text{estimated hours} - \text{actual hours}, 0)$.	Estimated remaining hours.
5. Recent velocity	Use recent logged hours over a 14-day lookback and convert to a weekly rate.	Recent hours per week.
6. Completion forecast	$\text{remaining hours} / \text{weekly velocity}$ gives estimated weeks remaining.	Forecast completion date.
7. Budget forecast	$\text{used hours} + \text{estimated remaining hours}$.	Forecast total hours and possible overrun.
8. Risk	Evaluate budget, schedule and task progress independently, then combine them.	HEALTHY, WARNING, AT_RISK or UNKNOWN.
9. Confidence	Check whether six evidence categories are sufficiently available.	LOW, MEDIUM or HIGH confidence.
10. Explanation	Give the calculated forecast to Gemini as evidence to explain, not to invent.	Summary, reasons, factors and recommendations.

A simplified data flow

Timesheets AI data	Forecast engine	Evidence / confidence	Manager UI
Projects Tasks Time entries Budget Dates	Progress Velocity Budget Schedule Risk	Available evidence Missing evidence GitHub / Jira	Forecast cards Charts Risk badges AI explanation

3. The evidence engine and confidence score

A prediction without context can look more certain than it really is. The evidence engine solves this by checking what information was actually available when the forecast was produced.

The forecast currently evaluates six evidence categories:

Evidence category	Available when...	Why it matters
PROJECT_DATES	Both project start and planned end dates exist.	Allows schedule and timeline progress comparisons.
BUDGET_HOURS	A positive hour budget exists.	Allows the system to calculate budget risk and overrun.
TASKS	The project contains tasks.	Provides measurable project progress.
TASK_ESTIMATES	Enough unfinished tasks have estimates.	Supports the remaining-work calculation.
RECENT_VELOCITY	Enough recent time has been logged.	Makes the completion-date estimate more representative.
EXTERNAL_EVIDENCE	GitHub or Jira evidence is available.	Adds supporting engineering activity evidence.

The manager sees both **available evidence** and **missing evidence**. For example, a forecast may say **3/6 evidence available**, then show exactly which three categories were available and which three were missing. This is more transparent than displaying a confidence label with no explanation.

Confidence rule

HIGH: at least 5 of 6 evidence categories available. **MEDIUM:** at least 3. **LOW:** fewer than 3.

Task estimate coverage is also exposed as a percentage. This is useful because a project may have tasks, but the forecast becomes less informative if many unfinished tasks have no estimate.

4. Risk analysis

The system does not reduce project health to one unexplained number. It calculates separate risk areas and then derives an overall status.

Risk area	What is checked	Why separate it?
Budget	Projected total hours compared with allocated budget hours.	A project can be on schedule but still consume too many hours.
Schedule	Forecast completion date compared with the planned end date.	A project can be under budget but late.
Task progress	Task completion compared with elapsed timeline progress where dates exist.	Detects a project that is progressing too slowly.
Overall	Combines the known risk areas.	Gives the manager a quick project-level signal.

The status can be **HEALTHY**, **WARNING**, **AT_RISK** or **UNKNOWN**. UNKNOWN is important: when the required data does not exist, the system does not pretend to know.

Why this is stronger than a dashboard metric

A normal dashboard tells the manager what has already happened. This feature uses the same operational data to estimate what may happen next, while still exposing the evidence and uncertainty behind that estimate.

5. GitHub, Jira and the AI explanation

External engineering evidence

The forecast can request supporting activity from the existing backend Evidence Engine. GitHub commit activity and Jira issue activity are kept separate from the core timesheet/task calculations. If an integration is unavailable, the forecast still completes instead of crashing.

Design decision	Benefit
External evidence is optional	Projects without Jira or GitHub can still receive a forecast.
Backend reuses existing integration/authentication logic	Avoids duplicating OAuth and integration rules in the Python service.
Evidence is supporting information	A GitHub commit does not automatically become 'hours worked' or falsely change task estimates.
Integration failure is handled gracefully	One unavailable service does not take down the forecasting feature.

What Gemini actually does

Gemini receives the structured forecast after the deterministic calculations are complete. Its job is to turn those results into a readable summary, explain the risk, identify contributing factors and produce manager recommendations. The prompt is designed to acknowledge missing or unavailable evidence instead of inventing values.

Why this matters for the 'AI is not a wow factor by itself' requirement

If Gemini were removed, the system would still calculate used hours, remaining work, velocity, completion date, budget projection, risk and confidence. Gemini improves interpretation; it is not the feature's calculation engine.

6. End-to-end architecture

The feature spans all three parts of the system rather than existing as an isolated AI endpoint.

Layer	What was added	Responsibility
AI service - FastAPI / Python	Forecast schemas, project forecast calculation, risk/confidence logic, evidence client, AI explanation, persistence support.	Performs the forecasting and produces a structured forecast.
Spring Boot backend	Project forecast DTOs, retrieval of latest persisted forecast, manual sync, evidence endpoint and project controller/service support.	Acts as the authenticated application boundary between frontend and AI service.
Shared PostgreSQL	Project forecast stored through the existing ai_insights approach.	Keeps the latest forecast available without recalculating on every page load.
Angular frontend	Forecast tab, models, service calls, cards, charts, evidence display, loading/error states and manual sync.	Presents the forecast to managers in an understandable form.

Persisted forecast vs. manual sync

Opening the Forecast tab retrieves the latest saved forecast. It does **not** automatically run the AI service again. When the manager chooses to sync, the system recalculates the forecast and then reloads the persisted version so the UI receives the authoritative last-synced timestamp.

Action	What happens	Why
Open Forecast tab	GET latest persisted project forecast.	Fast page load and no unnecessary AI call.
Manager presses Sync	POST manual forecast synchronization.	Fresh calculation only when requested.
After sync	Reload persisted forecast.	UI displays the saved result and correct lastSyncedAt value.

7. Benefits of the wow factor

Benefit	What it means in practice
Predictive rather than purely historical	Managers see an expected direction, not only hours that have already been logged.
Uses multiple sources of project evidence	The result combines project, task, time, budget, schedule and optional integration data.
Transparent confidence	Managers can see why confidence is high, medium or low and which evidence is missing.
Explainable risk	Budget, schedule and task-progress risk are visible separately.
Graceful uncertainty	Missing dates, estimates, velocity or integrations do not force fake values.
Efficient use of AI	Saved forecasts avoid recalculating every time the tab is opened.
Manager-focused	The feature is restricted to the role that needs project-level forecasting.
Actionable output	The explanation translates calculations into factors and recommendations.
Testable design	Deterministic helper functions can be unit tested independently from the generative AI layer.

The core 'wow'

Timesheets are normally retrospective: they tell you where time went. This feature turns timesheet and project data into a forward-looking delivery signal, then explains the quality of the evidence behind it.

8. Computational complexity

Complexity describes how the amount of work grows as the project contains more tasks, time entries and external evidence. The important point is that the forecasting logic is mostly linear; it does not compare every task with every other task.

Operation	Complexity	Explanation
Load / scan project tasks	$O(T)$	The forecast processes each task at most a small number of times.
Task completion calculation	$O(T)$	Counts completed tasks and loops through unfinished tasks.
Remaining-hours calculation	$O(T)$	Each task contributes at most once to the remaining-work total.
Task estimate coverage	$O(T)$	Checks unfinished tasks to determine how many have estimates.
Logged-hours aggregation	$O(E)$ conceptually	The database aggregates matching time entries. Actual DB cost depends on indexes and query planning.
Recent velocity aggregation	$O(E_r)$ conceptually	Aggregates recent matching time entries, where E_r is entries in the lookback window.
Budget / schedule / risk rules	$O(1)$	A fixed number of arithmetic operations and comparisons.
Evidence confidence	$O(T) + O(1)$	Task estimate coverage is linear; the six evidence-category checks are constant.
External evidence summary	$O(A)$	Depends on the number of returned activity records/categories.
AI explanation	Not expressed well as Big-O here	A remote model call is dominated by network/model latency and prompt/output token size rather than the local forecast algorithm.

Overall local forecast complexity: approximately $O(T + E + A)$ at the conceptual level, where T is the number of project tasks, E is the relevant time-entry workload processed by database aggregations, and A is external evidence returned. Database indexes can significantly reduce the practical cost of locating relevant rows.

Why this is good

The algorithm scales predictably. Doubling the number of tasks does not cause a quadratic explosion such as $O(T^2)$. Most calculations are single passes or database aggregates.

Space complexity

The calculation keeps a list of project tasks and a small structured result in memory. Ignoring ORM/database internals and external response sizes, the main in-process space requirement is approximately $O(T + A)$.

9. Why this qualifies as more than 'we integrated Gemini'

If it were only an AI integration...	What this implementation actually does
Send project text to Gemini and ask it to predict.	Calculates the forecast from structured operational data first.
Trust whatever the model says.	Uses deterministic formulas for hours, progress, velocity, schedule and risk.
Give one unexplained prediction.	Shows risk categories, confidence, available evidence and missing evidence.
Fail when an integration is missing.	Treats GitHub/Jira as optional supporting evidence.
Call AI every time the page opens.	Persists the forecast and provides explicit manual synchronization.
Mix AI output with source-of-truth data.	Keeps calculation, evidence and explanation as separate responsibilities.

The technical novelty for the project is therefore the **evidence-driven forecasting pipeline**: operational data is transformed into measurable progress, remaining effort, velocity, expected delivery, risk and confidence. Generative AI is the final explanation layer.

A concise demo explanation

What to say

"Our wow factor is not that we connected Gemini. We built a project forecasting engine on top of the timesheet system. It uses actual logged hours, task progress, estimates, recent velocity, budget and project dates to calculate where the project is heading. We also score the quality of the evidence, show managers what data was available or missing, and optionally use GitHub and Jira as supporting evidence. Gemini only explains the forecast that our system has already calculated."

10. What to demonstrate to the lecturer

A strong demonstration should make the calculation and evidence visible rather than focusing only on the UI.

Demo moment	What to point out
Open a project forecast	The page loads the latest persisted result rather than recalculating automatically.
Show used vs forecast vs budget	The forecast combines actual time with estimated remaining work.
Show task completion and velocity	Delivery date is connected to remaining work and recent team pace.
Show risk badges	Risk is broken into budget, schedule and task progress.
Show confidence	Explain the 6 evidence categories and the threshold for HIGH / MEDIUM / LOW.
Show available and missing evidence	The manager can inspect what supported the forecast instead of blindly trusting it.
Show GitHub/Jira evidence	External activity strengthens context but does not replace core project data.
Show AI explanation	Point out that Gemini explains calculated values rather than creating the forecast numbers.
Press Sync	Demonstrate intentional recalculation and persistence of the updated forecast.

Questions you should be ready to answer

- **Why is this AI?** The system combines deterministic forecasting with an AI explanation layer; the predictive engine itself is evidence-driven and does not depend on a language model guessing numbers.
- **What happens when data is missing?** The feature records missing evidence, lowers confidence where appropriate, uses UNKNOWN for risks that cannot be calculated, and continues where possible.
- **Why use recent velocity?** It reflects the team's recent pace rather than assuming a fixed productivity rate.
- **Why persist forecasts?** It reduces repeated AI/service calls, provides a stable result for the UI, and allows an explicit sync workflow.
- **Why include GitHub and Jira?** They provide supporting engineering evidence while remaining separate from timesheet hours and task estimates.
- **What is the complexity?** The core local algorithm is approximately $O(T + E + A)$, dominated by task processing and database aggregation rather than pairwise comparisons.

11. Current limitations and future improvements

Being clear about limitations makes the feature easier to defend technically. A forecast is an estimate, not a guarantee.

Current limitation	Possible future improvement
Velocity is based on a fixed recent lookback window.	Compare multiple windows or use weighted trends.
Task estimates depend on users entering useful estimates.	Add estimate-quality checks or historical calibration.
GitHub/Jira evidence is supporting evidence, not a direct productivity measure.	Develop richer evidence rules while avoiding simplistic commit-count productivity scoring.
Risk thresholds are rule-based.	Calibrate thresholds from historical project outcomes.
Forecast confidence is category-based.	Introduce weighted evidence quality once enough historical data exists.
Generative explanations may vary in wording.	Keep deterministic values as the source of truth and validate explanation output.

Final takeaway

The feature turns Timesheets AI from a system that records work into a system that can help a manager understand project trajectory. Its strongest technical contribution is the combination of deterministic forecasting, evidence-aware confidence, risk analysis, optional engineering evidence, persistence, and explainable AI.